

GapNav: Margin-Aware Gap Entry and Traversal Control for Legged Robots

Abstract—Legged robots now move reliably over rough terrain, stairs, rubble, and cluttered environments, but very tight passages remain difficult. The challenge is often not whether the robot can fit at the entrance, but whether it can keep enough side clearance throughout the crossing. Small yaw errors, lateral corrections, or body-height changes can quickly use up the limited remaining space. Standard navigation systems usually check the opening with a fixed body model and leave the rest to the low-level controller, which is often not enough when the clearance is only a few centimeters. We present GapNav, a passage-level controller for near-clearance traversal in legged robots. The key idea is to treat the remaining side margin as the quantity to estimate before entry and preserve during traversal. GapNav uses a motion-aware used-width model, a conservative entry rule under bounded uncertainty, and a dedicated passage mode that adjusts motion after entry to reduce unnecessary margin loss. We evaluate GapNav in Isaac Sim across multiple environments with varied passage layouts, and on a physical quadruped in real-world narrow-passage trials. In simulation, GapNav reaches $88.1 \pm 1.7\%$ success with 0.74 ± 0.09 cm margin error while improving all main metrics over the baselines. On hardware, it achieves 46.7% success even at 2 cm side margin per side, and performance improves as more clearance becomes available. These results show that explicit margin estimation and preservation can improve near-clearance legged traversal.

I. INTRODUCTION

Legged robots have improved rapidly in recent years. They can move over rough terrain, climb stairs, recover from disturbances, and operate in places that are still difficult for wheeled platforms [1], [2], [3], [4]. These advances have made legged systems more practical for inspection, field robotics, and disaster-response settings, where uneven ground and cluttered layouts are common [5], [6], [7]. However, very tight passages remain difficult when the opening is only slightly wider than the robot body [5], [7], [8], [9]. In these cases, the challenge is not only whether the robot fits at the entrance. What matters just as much is what happens after the robot starts moving through the opening. A small yaw error, a lateral correction, or a body-height adjustment can quickly use up the little free space that remains, making contact or failure much more likely [7], [9]. Standard navigation systems usually check the opening with a fixed body model and then leave the rest to the low-level controller [5], [6], [7]. That approximation is often too coarse when only a few centimeters of clearance remain.

Prior work has addressed important parts of this problem. Planning methods for confined spaces have shown that body pose and geometry matter when clearance is limited [5], [6]. Learning-based locomotion methods have shown that legged robots can acquire strong behaviors in narrow, cluttered, and constrained settings [7], [8]. Safety-aware control methods

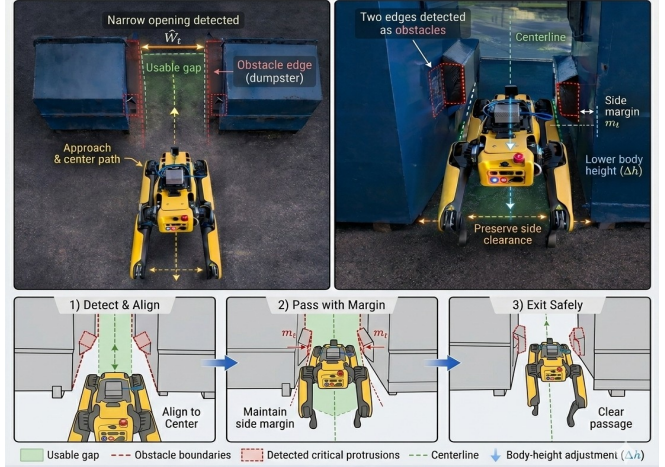


Fig. 1: GapNav in a near-clearance traversal scenario. The robot approaches a narrow opening, estimates usable width and remaining side margin from local sensing, and enters only when a conservative margin check indicates feasible passage. During traversal, GapNav shapes the command to preserve lateral clearance and avoid contact.

have also shown that feasibility and geometric constraints can be handled directly inside the control loop [9], [10], [11]. Still, one practical question remains open: once a robot starts moving through a very tight opening, how should it estimate and preserve the small amount of side margin that remains during the crossing?

We study *near-clearance passage traversal* for legged robots, where success depends on keeping enough side margin from entry to exit. Instead of treating the robot as a fixed-width body, we model the width it uses during motion. In a tight passage, that used width changes with body orientation, lateral correction, body height, and the command being executed. This leads to a more useful control view of the problem: the system must estimate how much usable width is available before entry, predict how much width the robot is likely to use while crossing, and preserve enough remaining side margin until the robot clears the opening.

To address this, we present **GapNav**, a passage-mode control layer that sits between high-level navigation and the underlying locomotion controller. GapNav does not replace the low-level gait controller. Instead, it adds a compact passage model, a conservative entry rule under bounded uncertainty, and a passage-mode controller that limits unnecessary margin loss once the robot enters the opening. This keeps the method lightweight and easy to pair with an existing locomotion stack, while leaving balance, gait

timing, and contact handling to the platform-specific low-level controller [12], [1], [3].

Main Contributions:

- **A margin based formulation for near clearance traversal.** We formulate near clearance passage traversal around the side margin that remains during the crossing, rather than checking a fixed body width only at the entrance. This gives the system a more useful way to represent whether a tight crossing remains feasible.
- **A motion aware model of used width.** We introduce a compact model that captures how body orientation, lateral correction, body height, and commanded motion change the width used by the robot inside a narrow passage. This allows the controller to account for how corrective actions affect the free space that remains.
- **A conservative entry rule for narrow passages.** We derive a simple rule that integrates the measured opening width with bounded sensing and tracking error, so the robot enters only when the available opening still leaves enough usable side margin for the crossing.
- **A dedicated passage mode controller.** We design a control layer for tight passages that regulates forward motion, lateral correction, turning, and body adjustment so the robot can continue through the opening while preserving side margin during traversal.

We evaluate GapNav across multiple Isaac Sim environments with varied passage layouts and on a physical quadruped in real world narrow passage trials.

II. RELATED WORK

A. Legged locomotion in narrow and confined spaces

Research on legged locomotion in narrow and confined spaces has progressed along both planning and learning directions. On the planning side, Buchanan *et al.* showed that posture and body shape need to be part of the planning problem when clearance is limited. They introduced posture adaptation during mapping and planning for confined spaces [5], and later extended this idea to perceptive whole body planning with online body pose, footholds, and swing motion [6]. These works showed that a fixed body model is often too crude near obstacles. On the learning side, Tidd *et al.* learned a dedicated narrow gap behavior together with a policy that decides when to use it [7]. Miki *et al.* used hierarchical reinforcement learning and 3D scene representations to adapt posture in confined and rough environments [8]. Xu *et al.* studied dexterous legged locomotion in confined 3D spaces with a hierarchical controller that couples waypoint planning with reinforcement learning based local motion [13]. Guo *et al.* studied quadruped locomotion for narrow pipe inspection, where staying centered inside a tight structure is central to the task [14]. More recent learned systems also show that a single control stack can handle crouching, jumping, and obstacle crossing across varied terrain [15], [16]. Together, these works show that legged robots can acquire strong behaviors in constrained environments. What remains less studied is the passage level problem in very tight openings,

where a crossing may appear feasible at entry but fail when small corrections use up the remaining side margin.

B. Perception and terrain aware locomotion

Perception has become central to modern legged locomotion, especially when footholds and body pose need to adapt to local terrain structure. Prior work has shown benefits from vision based and map based strategies for selecting footholds, adjusting posture, and improving robustness on rough terrain and cluttered scenes [2], [17], [18], [19], [15], [16], [20]. Elevation map pipelines also provide a practical geometric representation that supports both navigation and perceptive locomotion [21]. Other learning systems address confined motion more directly by training policies that use richer 3D scene representations or posture adaptation for tight spaces [7], [8], [14]. These approaches improve terrain understanding and locomotion robustness, but they are not usually framed around the question that matters most in near clearance passages: how much side margin is left, and how quickly yaw and lateral corrections use it up after entry. GapNav builds on local sensing and compact map context, but turns them into an explicit passage variable, the remaining side margin, which is then used to decide entry and regulate corrective actions during traversal.

C. Safety, feasibility, and tracking in constrained locomotion

A separate line of work brings collision handling and constraint satisfaction into the control stack. Liao *et al.* tackled narrow space locomotion through a safety critical NMPC framework with obstacle constraints, targeting tight real world navigation for quadrupeds [9]. Grandia *et al.* combined barrier functions with MPC in a layered safety design for legged locomotion, emphasizing constraint enforcement alongside dynamic stability [10]. Abdalla *et al.* studied feasibility guarantees by constructing efficient feasible regions under balance, actuation, and kinematic limits [11]. In parallel, Gaertner *et al.* presented a collision free MPC that incorporates environment collision costs and demonstrates obstacle avoidance in static and dynamic scenes [22]. Finally, learned tracking layers such as DTC show how a policy can provide robust tracking of planned motions on difficult terrain [23]. These methods are important steps toward principled collision avoidance and feasibility during locomotion, but near clearance passages still pose a practical control problem: a crossing can satisfy constraints at the entrance and still fail moments later because a small correction spends the last available slack. GapNav addresses that specific gap with a passage level structure that separates (i) a conservative entry decision from (ii) control after entry, and uses the remaining side margin as the central variable for passage estimation and traversal control.

III. GAPNAV CONTROLLER

This section presents the proposed controller for near-clearance gap traversal. An overview of the proposed method is shown in Fig. 2. GapNav adds a passage-level decision and control layer between the navigation stack and the low-level locomotion controller.

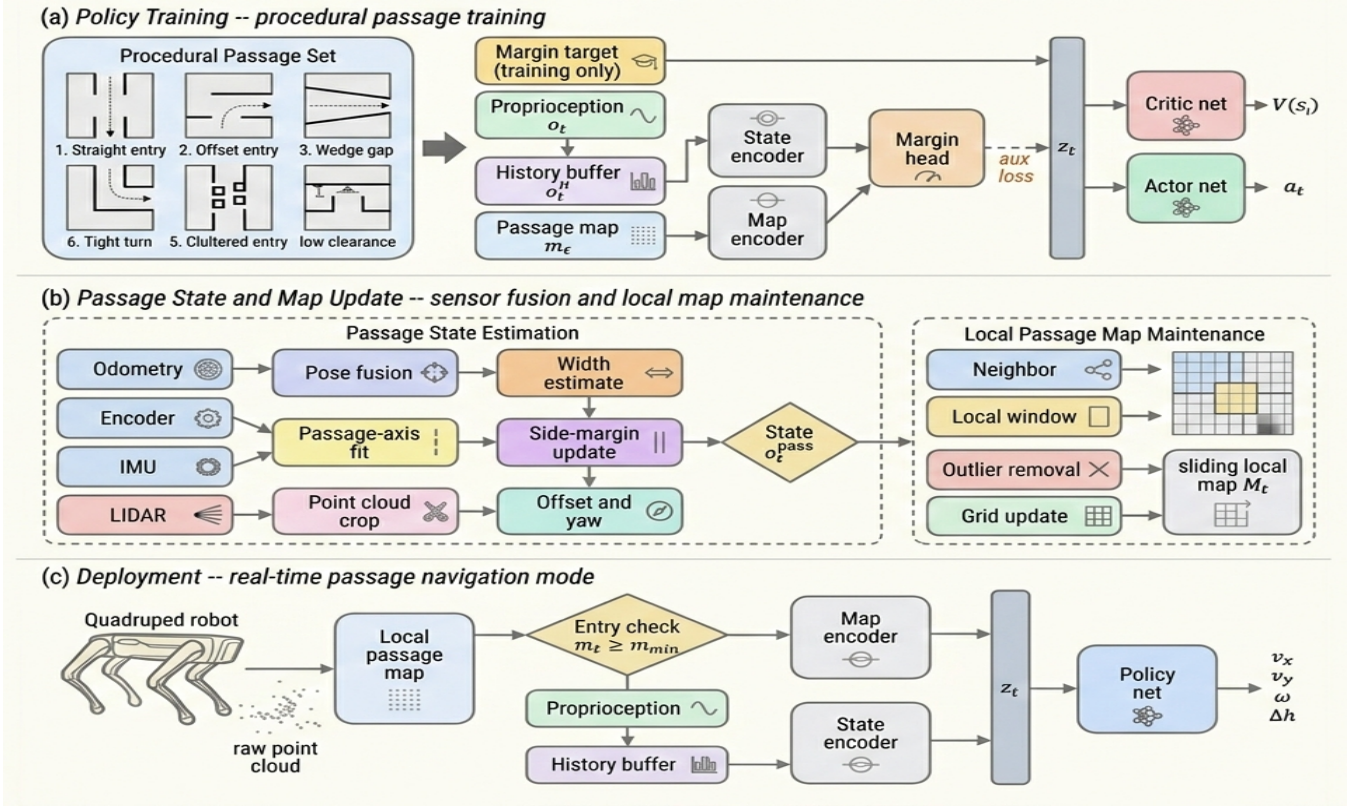


Fig. 2: Overview of the GapNav framework.

A. Problem formulation

We consider a legged robot that approaches a passage whose width is only slightly larger than the width used by the robot during motion. The key difficulty is that the opening may appear feasible at the entrance, but the crossing can still fail after small yaw corrections, small lateral re-centering motions, or small body-height changes reduce the side margin that remains. We therefore model passage traversal as a partially observed control problem in which the controller must reason about both the local free space and the robot's changing occupied width during execution.

At each time step, the robot receives proprioceptive observations, a short history buffer, and a robot-centered local passage map built from onboard sensing. Let the latent physical state be $s_t \in \mathcal{S}$, the observation be $o_t \in \mathcal{O}$, and the action be $a_t \in \mathcal{A}$. The controller seeks a policy $\pi_\theta(a_t | o_t)$ that maximizes the expected discounted return

$$J(\pi_\theta) = \mathbb{E} \left[\sum_{t=0}^{\infty} \gamma^t r_t \right], \quad (1)$$

where $\gamma \in (0, 1)$ is the discount factor and r_t is the task reward.

B. Observations, actions, and occupied width model

Observation. We use a compact observation that separates body state from passage geometry. The body-state term is

$$o_t^{\text{body}} = [\omega_t, g_t, q_t, \dot{q}_t, a_{t-1}], \quad (2)$$

where ω_t is body angular velocity, g_t is the gravity vector projected into the body frame, q_t and $\dot{q}_t$ are joint positions and velocities, and a_{t-1} is the previous action. To improve robustness under short sensing dropouts and delayed corrections, we use a short history buffer

$$o_t^H = [o_t^{\text{body}}, o_{t-1}^{\text{body}}, \dots, o_{t-H}^{\text{body}}]. \quad (3)$$

The passage-state term is

$$o_t^{\text{pass}} = [\hat{w}_t, \hat{y}_t, \hat{\psi}_t, \hat{d}_t^{\text{in}}, \hat{d}_t^{\text{out}}, \hat{h}_t, \hat{m}_t], \quad (4)$$

where $\hat{w}_t$ is the estimated opening width, $\hat{y}_t$ is the lateral offset from the local centerline, $\hat{\psi}_t$ is the yaw error with respect to the passage axis, $\hat{d}_t^{\text{in}}$ and $\hat{d}_t^{\text{out}}$ are distances to the entrance and exit, $\hat{h}_t$ is the available overhead margin, and $\hat{m}_t$ is the current remaining side margin estimate. In parallel, a robot-centered local passage map M_t stores the nearby free-space layout around the entrance and the current body position. The full observation is the tuple

$$o_t = (o_t^H, o_t^{\text{pass}}, M_t). \quad (5)$$

Action. The controller outputs high-level commands that are tracked by the platform's underlying locomotion stack:

$$a_t = [v_{x,t}, v_{y,t}, \omega_t, \Delta h_t]. \quad (6)$$

Here $v_{x,t}$ and $v_{y,t}$ are desired forward and lateral body velocities, ω_t is the desired yaw rate, and Δh_t is a body-height adjustment. This action interface keeps the method applicable

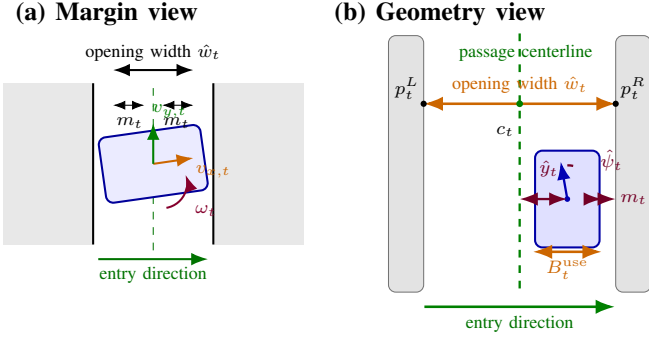


Fig. 3: Passage geometry used by GapNav. **(a) Margin view.** The controller reasons about the remaining side margin m_t rather than only the nominal body width, and shapes the motion inside the passage so the available clearance is not used up too quickly. **(b) Geometry view.** The controller identifies the left and right passage boundaries, measures the opening width $\hat{w}_t$, defines a local passage frame, and computes the lateral offset $\hat{y}_t$, yaw error $\hat{\psi}_t$, used width B_t^{use} , and remaining side margin m_t .

across legged platforms while leaving contact timing and low-level stabilization to the robot's native controller.

Occupied width during motion. The controller should not reason only about the nominal body width. During a very tight crossing, the width used by the robot changes with orientation, lateral motion, and body-height adjustment. We model the width used during motion as

$$B_t^{\text{use}} = B_0 + \alpha_\psi |\hat{\psi}_t| + \alpha_y |v_{y,t}| + \alpha_\omega |\omega_t| + \alpha_h |\Delta h_t|, \quad (7)$$

where B_0 is the nominal body width term and the coefficients $\alpha_\psi, \alpha_y, \alpha_\omega, \alpha_h$ capture how heading error and corrective motion consume lateral clearance. This linear form is simple enough for online use and can be calibrated from simulation rollouts and a small number of hardware trials. The remaining side margin is then

$$m_t = \frac{\hat{w}_t - B_t^{\text{use}}}{2}. \quad (8)$$

A positive margin means the crossing remains feasible under the current command. A negative margin means the robot is using more width than the measured opening allows.

C. Local Passage Geometry and State Update

GapNav uses a local geometric description of the opening before the robot enters the passage. Rather than treating the opening as a single binary obstacle check, the controller builds a local passage frame and updates a compact set of measurements that describe how much usable side margin remains (as shows in Fig. 3).

At each control step, we crop the local point cloud to a region centered on the candidate opening and aligned with the current approach direction. From this cropped observation, we extract the two boundary segments that define the left and right sides of the passage near the entrance. Let $p_t^L, p_t^R \in \mathbb{R}^2$

denote the closest points on the left and right boundaries at the entrance slice. The measured opening width is then

$$\hat{w}_t = \|p_t^R - p_t^L\|_2. \quad (9)$$

The midpoint of the opening defines the local passage center,

$$c_t = \frac{1}{2} (p_t^L + p_t^R). \quad (10)$$

We use the normalized cross-passage direction

$$\hat{n}_t = \frac{p_t^R - p_t^L}{\|p_t^R - p_t^L\|_2}, \quad (11)$$

and define the local passage axis as the orthogonal direction

$$\hat{a}_t = R_{\pi/2} \hat{n}_t, \quad (12)$$

where $R_{\pi/2}$ rotates a vector by 90° in the plane. This gives a local frame aligned with the passage: $\hat{n}_t$ points across the opening, and $\hat{a}_t$ points along the passage.

Let $b_t \in \mathbb{R}^2$ be the robot base position projected onto the ground plane, and let θ_t be the robot yaw angle. Using the local frame above, we compute the lateral offset from the passage centerline as

$$\hat{y}_t = \hat{n}_t^\top (b_t - c_t), \quad (13)$$

and the yaw error relative to the passage axis as

$$\hat{\psi}_t = \text{wrap}(\theta_t - \text{atan2}(\hat{a}_{t,y}, \hat{a}_{t,x})), \quad (14)$$

where $\text{wrap}(\cdot)$ maps the angle to $(-\pi, \pi]$.

These measurements define the geometric state used by the entry rule and the passage-mode controller. In particular, the controller uses $\hat{w}_t$ to estimate the available opening width, $\hat{y}_t$ to measure how far the robot is from the passage centerline, and $\hat{\psi}_t$ to determine how strongly the robot is misaligned with the opening. Together with the current body-height adjustment and short-horizon motion history, these terms are sufficient to estimate the width the robot is likely to use during the crossing. To reduce sensitivity to sensor noise and small point-cloud discontinuities near the entrance, we apply a short temporal filter to the geometric measurements,

$$\bar{z}_t = \lambda \bar{z}_{t-1} + (1 - \lambda) z_t, \quad (15)$$

where $z_t \in \{\hat{w}_t, \hat{y}_t, \hat{\psi}_t\}$ and $\lambda \in [0, 1)$. The filtered values are then passed to the entry rule and to the passage-mode controller.

D. Passage Entry Rule and Passage-Mode Control

Once the local passage geometry has been estimated, the controller decides whether the opening should be attempted and, if so, how the robot should move through it. The decision is based on a conservative estimate of the side margin that will remain after accounting for sensing uncertainty and motion-dependent width usage.

Let B_t^{use} denote the predicted width used by the robot during the crossing. We model this quantity as

$$B_t^{\text{use}} = B_0 + \alpha_\psi |\hat{\psi}_t| + \alpha_y |v_{y,t}| + \alpha_\omega |\omega_t| + \alpha_h |\Delta h_t|, \quad (16)$$

where B_0 is the nominal body width, and the remaining terms capture how yaw misalignment, lateral correction, turning, and body-height adjustment increase the width used in practice. This form is intentionally simple: it does not attempt to model full-body contact geometry, but it captures the main ways in which control actions consume the limited clearance available inside a tight passage. Using the measured opening width $\hat{w}_t$ and the predicted used width above, we define the conservative remaining side margin as

$$m_t = \frac{\hat{w}_t - 2\epsilon_w - B_t^{\text{use}} - 2\epsilon_u}{2}, \quad (17)$$

where ϵ_w bounds the opening-width estimation error and ϵ_u bounds the mismatch between predicted and realized used width. The robot enters the passage only if

$$m_t \geq m_{\min}, \quad (18)$$

where $m_{\min}$ is the minimum side margin required for a feasible crossing. This rule does not try to certify full safety in a formal sense. Its purpose is more practical: it prevents the robot from entering when the measured opening leaves too little usable width to tolerate even a small correction. Once the condition in (18) is satisfied, the controller switches into passage mode. In this mode, the policy outputs the command

$$a_t = (v_{x,t}, v_{y,t}, \omega_t, \Delta h_t), \quad (19)$$

where $v_{x,t}$ is the forward velocity, $v_{y,t}$ is the lateral correction, ω_t is the yaw rate, and Δh_t is the body-height adjustment. These commands are then filtered by a lightweight shaping rule before they are sent to the low-level locomotion controller. The forward command is reduced when the robot is strongly misaligned or when the remaining margin is small.

$$v_{x,t}^* = v_{x,t} \cdot \text{clip}\left(\frac{m_t - m_{\min}}{m_{\text{ref}} - m_{\min}}, 0, 1\right), \quad (20)$$

where $m_{\text{ref}} > m_{\min}$ is a reference margin at which full forward speed is allowed. Lateral correction and yaw rate are also clipped so that a single corrective step does not spend too much of the remaining width,

$$|v_{y,t}^*| \leq \bar{v}_y(m_t), \quad |\omega_t^*| \leq \bar{\omega}(m_t), \quad (21)$$

where $\bar{v}_y(\cdot)$ and $\bar{\omega}(\cdot)$ are monotone increasing limits. When the remaining side margin is small, the controller allows only mild re-centering and turning. As the passage becomes less restrictive, these limits are relaxed.

The body-height command is handled in the same way. If overhead adjustment is needed, the controller restricts Δh_t to a range that preserves lateral feasibility,

$$|\Delta h_t^*| \leq \bar{h}(m_t), \quad (22)$$

so that height changes do not trigger an unnecessarily large increase in used width at the same time as a lateral correction. The entry rule determines whether the opening leaves enough usable width to justify an attempt, while the passage-mode controller regulates how quickly that width is spent once the robot enters. In this way, GapNav separates two parts of the

Algorithm 1 GapNav passage-mode

```

1: mode  $\leftarrow$  NAV
2: while goal not reached do
3:   Update local cloud, map  $M_t$ , state  $s_t$ 
4:   Detect passage candidate  $\mathcal{P}_t$  on route
5:   if mode = NAV and  $\mathcal{P}_t$  exists then
6:     mode  $\leftarrow$  APPROACH
7:   end if
8:   if mode  $\in$  {APPROACH, CROSS} and  $\mathcal{P}_t$  exists then
9:     Estimate  $(\hat{w}_t, \hat{y}_t, \hat{\psi}_t)$ , compute  $B_t^{\text{use}}$  and  $m_t$ 
10:  end if
11:  if mode = APPROACH then
12:    mode  $\leftarrow$   $\begin{cases} \text{CROSS}, & m_t \geq m_{\min} \\ \text{NAV}, & \text{otherwise} \end{cases}$ 
13:  end if
14:  if mode = CROSS then
15:    Send  $\tilde{a}_t = \text{SHAPECLIP}(\pi_\theta(s_t, M_t), m_t)$  to controller
16:    if  $m_t < m_{\text{abort}}$  or stall then
17:      mode  $\leftarrow$  RECOVER
18:    else if  $d_t^{\text{out}} \leq d_{\text{exit}}$  then
19:      mode  $\leftarrow$  NAV
20:    end if
21:  end if
22:  if mode = RECOVER then
23:    Stop/back out/replan; mode  $\leftarrow$  NAV
24:  end if
25: end while

```

problem that are often mixed together in practice: deciding whether the opening is still feasible and controlling the crossing so that it remains feasible until exit. The full mode-switching logic of GapNav is summarized in Algorithm 1.

Proposition 1 (Conservative entry): Assume $|w - \hat{w}| \leq \epsilon_w$ and $|B_{\text{use}}(x_0, u_0) - \hat{B}_{\text{use}}(x_0, u_0)| \leq \epsilon_b$. If

$$\hat{w} - \epsilon_w \geq \hat{B}_{\text{use}}(x_0, u_0) + \epsilon_b + 2m_{\min},$$

then the true side margin at entry satisfies

$$m_0 = \frac{1}{2}(w - B_{\text{use}}(x_0, u_0)) \geq m_{\min}.$$

Proof. Since $w \geq \hat{w} - \epsilon_w$ and $B_{\text{use}}(x_0, u_0) \leq \hat{B}_{\text{use}}(x_0, u_0) + \epsilon_b$,

$$w - B_{\text{use}}(x_0, u_0) \geq \hat{w} - \epsilon_w - \hat{B}_{\text{use}}(x_0, u_0) - \epsilon_b \geq 2m_{\min}.$$

Dividing by two gives $m_0 \geq m_{\min}$. ■

Proposition 2 (Margin preservation): Let

$$m_t = \frac{1}{2}(w_{\text{safe}} - B_{\text{use},t}).$$

If

$$B_{\text{use},t} \leq w_{\text{safe}} - 2m_{\min} \quad \forall t,$$

then

$$m_t \geq m_{\min} \quad \forall t.$$

Proof. Substituting the bound on $B_{\text{use},t}$ gives

$$m_t \geq \frac{1}{2}(w_{\text{safe}} - (w_{\text{safe}} - 2m_{\min})) = m_{\min}. \quad \blacksquare$$

E. Network Design and Training

The controller is implemented as a compact actor-critic policy with an auxiliary branch that predicts the remaining side margin. The goal of this design is not to build a large perception stack, but to give the policy just enough structure to combine local passage geometry with short-horizon motion history.

The input to the network has two parts. The first part is a low-dimensional state vector that includes proprioceptive features, the filtered geometric measurements from Sec. III-C, and recent motion history. In our implementation, this state includes the current body velocity, yaw rate, body-height command, filtered opening width, lateral offset, yaw error, and a short history buffer of recent commands and observations. The second part is the local passage map, which provides a compact spatial description of the opening near the entrance. The state vector is processed by a state encoder, while the local passage map is processed by a separate map encoder. The encoded features are then fused into a shared latent representation,

$$z_t = f_{\text{fuse}}(f_{\text{state}}(s_t), f_{\text{map}}(M_t)), \quad (23)$$

where s_t denotes the low-dimensional state, M_t denotes the local passage map, $f_{\text{state}}(\cdot)$ is the state encoder, $f_{\text{map}}(\cdot)$ is the map encoder, and $f_{\text{fuse}}(\cdot)$ is the feature fusion module. From this shared latent representation, the network produces three outputs. The actor predicts the passage-mode command,

$$a_t = \pi_\theta(z_t), \quad (24)$$

the critic predicts the state value,

$$V_\phi(z_t), \quad (25)$$

and the auxiliary branch predicts the minimum side margin expected over the near future,

$$\hat{m}_t^{\text{pred}} = g_\eta(z_t). \quad (26)$$

This auxiliary prediction is not used as a separate planner. Instead, it helps the network learn a representation that better tracks how close the robot is to using up the remaining width during the crossing.

We train the policy with PPO [24] and use generalized advantage estimation to compute the return targets. During simulation, the policy runs at a lower rate than the low-level stabilizing controller. The policy updates the high-level command at 5 Hz, while the simulator advances at 50 Hz and the internal controller tracks the commanded body velocity and body height at the faster loop rate. This separation keeps the policy focused on passage-level motion decisions and leaves fast stabilization to the underlying controller.

The reward is designed to encourage forward progress while discouraging unnecessary corrections and unsafe entries. At each step, we use

$$\begin{aligned} r_t = & w_p \Delta d_t - w_y |\hat{y}_t| - w_\psi |\hat{\psi}_t| \\ & - w_m \max(0, m_{\text{safe}} - m_t) \\ & - w_u \|a_t\|_2^2 - w_c \mathbb{I}[\text{contact}], \end{aligned} \quad (27)$$

where Δd_t is forward progress along the passage axis, m_t is the conservative remaining side margin from (17), and $\mathbb{I}[\text{contact}]$ is an indicator for trunk or hip contact. The reward therefore balances three competing goals: making progress, keeping the robot centered and aligned, and avoiding actions that use up the available width too aggressively.

The auxiliary margin-prediction branch is trained with a regression loss on the realized minimum side margin observed over a short future window,

$$\mathcal{L}_{\text{margin}} = \left\| \hat{m}_t^{\text{pred}} - m_t^{\text{target}} \right\|_2^2, \quad (28)$$

where m_t^{target} is computed from the realized minimum side margin over the next few control steps in simulation. The full training objective combines the PPO loss with this auxiliary term,

$$\mathcal{L} = \mathcal{L}_{\text{PPO}} + \lambda_m \mathcal{L}_{\text{margin}}, \quad (29)$$

where λ_m controls the influence of the auxiliary branch. The actor and critic are implemented as two-layer multilayer perceptrons with 128 hidden units per layer. The auxiliary margin-prediction branch uses the same fused latent representation, followed by a smaller output head. This compact architecture reflects the intended role of GapNav as a lightweight passage-level controller that can be trained efficiently in simulation and layered on top of an existing low-level locomotion stack.

F. Deployment and Integration

GapNav is designed as a passage-level controller that sits between the global navigation stack and the low-level locomotion controller. During normal motion, the robot follows its standard navigation policy. When the local map indicates a candidate narrow opening along the planned route, the system switches to the local passage frame described in Sec. III-C and evaluates the entry rule in Sec. III-D. If the opening does not leave enough usable side margin, control remains with the nominal navigation stack and the robot either waits, re-approaches, or requests a local replan. If the entry condition is satisfied, the controller switches into passage mode.

In passage mode, the policy produces the high-level command $(v_x, v_y, \omega, \Delta h)$ at the slower policy rate. These commands are shaped by the margin-preserving rules in Sec. III-D and then sent to the underlying locomotion controller, which handles balance, contact timing, and fast stabilization. The passage mode remains active until one of three conditions is met: the robot clears the exit, the conservative remaining side margin drops below the minimum threshold, or the robot stops making forward progress for longer than a fixed timeout.

IV. EXPERIMENTAL EVALUATION

We evaluate GapNav around three questions. First, does it improve traversal in very narrow passages where small alignment and control errors quickly consume the remaining clearance? Second, do the entry rule and the margin model improve entry quality and side-margin estimation?

Third, does the same high-level passage controller transfer to hardware with the same control interface?

A. Experimental Setup

Fig. 4 shows the two evaluation settings. In simulation, we use Isaac Sim to generate narrow-passage layouts with controlled changes in side margin, entry offset, wall shape, and local clutter. Each episode begins from a nominal standstill pose between 1.2 m and 1.8 m from the entrance. The controller receives proprioception, a short history buffer, and the local passage map described in Sec. III-C. A traversal is counted as successful if the robot clears the exit without trunk or hip contact and without stalling for more than 2.0 s. For hardware validation, we use the same high-level passage-mode policy on a physical quadruped equipped with onboard LiDAR and proprioception. The passage rig has calibrated wall spacing and supports repeated trials over multiple entry offsets. The method itself is not tied to one robot. The only platform-specific part is the low-level locomotion controller that tracks $(v_x, v_y, \omega, \Delta h)$.

Metrics. We report five metrics. *Success rate* measures the fraction of traversals that clear the exit without a stop. *Entry precision* measures the fraction of committed entries that remain feasible at the start of the crossing, so it reflects the quality of the entry decision rather than the full outcome. *Contact rate* counts trunk or hip contacts per 100 traversals. *Crossing time* measures time from entry to exit. *Margin error* is the absolute difference between the predicted minimum side margin and the realized minimum side margin in simulation.

Baselines. We compare the full method against four baselines. **Centering only** uses the same action space but does not model motion-dependent used width. **Without entry rule** always enters once the planner reaches the approach pose. **Without width head** uses the passage map but does not predict a width correction. **Without map encoder** uses proprioception and history only. Together, these baselines separate the effect of the entry rule, the width-prediction branch, and the local passage representation.

B. Simulation Results

Fig. 5 shows the training behavior in simulation. GapNav reaches the highest return and remains the most stable in the later stage of training. Removing the map encoder slows convergence because the controller loses direct local geometric information. Removing the width head causes a larger drop in final reward, especially after the early stage, which shows that explicit width correction becomes more important once the policy reaches the harder low-clearance cases. The variant without the entry rule is the least stable among the structured baselines because it continues to enter passages that should have been rejected. Table I provides the main simulation result. GapNav is best on all reported metrics. More importantly, success, entry precision, and contact rate improve together. This matters because success can often be increased by entering more aggressively, but that usually raises contact. Here, GapNav improves the entry decision

and also uses the available clearance more effectively after entry. The largest gains appear in the tightest passages. At 2 cm of side margin per side, GapNav reaches 58% success, compared with 39% without the map encoder, 31% without the width head, and 24% without the entry rule. At 4 cm per side, the same ordering remains, but the gap narrows as the passage becomes less restrictive. As the available clearance increases, small control errors are easier to absorb, so explicit side-margin control becomes less important.

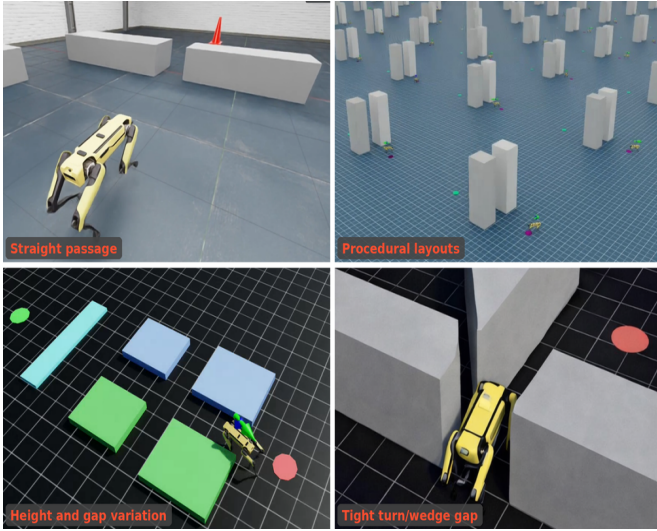
C. Ablation and Sensitivity Analysis

GapNav is built around a simple idea: the remaining side margin should be estimated before entry and preserved during traversal, rather than treated as a one-time check at the entrance. Fig. 6 shows this directly by comparing the predicted and realized minimum side margin. GapNav stays closest to the ideal diagonal and shows the smallest spread, indicating the lowest bias among the baseline variants. The figure also clarifies the role of the learned components. The map encoder improves the local representation of the opening, while the width head turns that representation into a correction for the width used during motion. Removing either increases margin error, with the larger drop appearing when the width head is removed.

The entry rule improves a different part of the behavior. Its effect appears most clearly in entry precision and contact rate. Without the entry rule, the controller enters too often when the opening is only marginally feasible, which lowers entry precision and sharply increases contact even though the downstream policy is unchanged. This supports the separation in GapNav between deciding when to enter and controlling motion after entry. Table II shows that the method remains stable under moderate miscalibration of the used-width coefficients and the uncertainty bounds in the entry rule. Small coefficient changes cause only modest drops in success and small increases in contact. Increasing the uncertainty bounds makes the controller more conservative and mainly reduces the number of attempted entries. Reducing them has the opposite effect: contact rises because the controller allows entries that should have been rejected. This is consistent with the role of the entry rule as a conservative filter rather than a hard guarantee.

D. Hardware Transfer and Practical Behavior

Table III shows that the same high-level passage policy transfers to hardware while keeping the same overall ordering seen in simulation. The largest drop appears in the tightest 2 cm setting, where sensing noise, small tracking delays, and small map bias have the greatest effect on the remaining side margin. By 6 cm per side, success rises to 88.9%, which closely follows the simulation trend. Fig. 7 helps explain this transfer behavior. In representative trials, the full method reduces lateral offset and yaw more quickly and keeps the remaining side margin above the safety threshold throughout the crossing. The important point is not only that the robot moves through the opening, but that the same passage-level



(a) Simulation



(b) Real-world

Fig. 4: **Evaluation setup.** (a) Isaac Lab/Sim training and test scenes spanning varied passage geometries. (b) Real-world validation on a 60cm narrow-gap approach and a 58cm narrow-bridge crossing.

TABLE I: Aggregate results on the held-out Isaac Sim benchmark. Numbers are mean $\pm$ standard deviation over 3 seeds and 960 evaluation episodes per method. Contact rate is measured per 100 traversals. Margin error is the absolute difference between predicted and realized minimum side margin.

Method	Success rate (%)	Entry precision (%)	Contact rate	Crossing time (s)	Margin error (cm)
GapNav (full)	88.1 ± 1.7	93.2 ± 1.3	8.7 ± 0.9	4.62 ± 0.18	0.74 ± 0.09
Without entry rule	71.4 ± 2.3	78.5 ± 2.0	21.3 ± 1.4	5.08 ± 0.24	1.26 ± 0.14
Without width head	76.8 ± 2.0	84.7 ± 1.6	15.9 ± 1.2	5.36 ± 0.22	1.61 ± 0.16
Without map encoder	80.4 ± 1.9	87.9 ± 1.7	13.7 ± 1.1	4.98 ± 0.21	1.29 ± 0.13
Centering only	68.9 ± 2.5	74.2 ± 2.1	23.8 ± 1.6	5.81 ± 0.27	1.94 ± 0.19

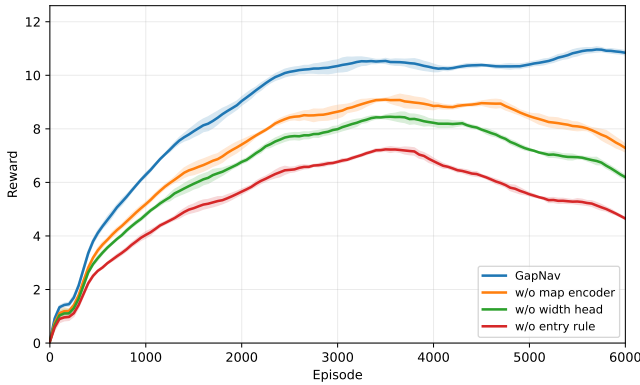


Fig. 5: Average training reward of GapNav and its baselines in Isaac Sim over 6000 episodes with 4096 parallel environments. Shaded regions indicate the standard deviation across three random seeds.

logic remains useful after moving from simulation to onboard sensing and a real low-level controller.

V. CONCLUSION AND FUTURE WORK

We presented GapNav, a passage-mode controller for legged robots aimed at near-clearance crossings where only a small side margin remains. The key idea is to estimate that

TABLE II: Sensitivity to moderate miscalibration on the held-out Isaac Sim benchmark. Values show the change relative to the nominal calibrated setting.

Perturbation	Δ Success (%)	Δ Contact /100
$\alpha_{\bullet} \times 0.9$	-1.8	+0.9
$\alpha_{\bullet} \times 1.1$	-1.2	+0.5
$\alpha_{\bullet} \times 0.8$	-4.6	+2.7
$\alpha_{\bullet} \times 1.2$	-3.9	+2.1
$\epsilon_w, \epsilon_u + 0.5$ cm	-1.5	-0.6
$\epsilon_w, \epsilon_u - 0.5$ cm	-2.4	+1.8
$\epsilon_w, \epsilon_u + 1.0$ cm	-3.7	-1.1
$\epsilon_w, \epsilon_u - 1.0$ cm	-6.2	+4.3

margin before entry and preserve it throughout the crossing. GapNav combines a motion-aware used-width model, a practical entry rule under bounded error, and a dedicated controller for passage traversal. In the tightest passages, where a few centimeters can determine whether a small correction is recoverable, GapNav improves success, entry precision, and contact rate, while also producing more accurate estimates of the minimum side margin. The method still has clear limits: performance drops when clutter distorts the local map, when the passage narrows late and unevenly, or when slip changes the link between commanded motion and the width used by the robot. We plan to extend the same passage-level reasoning to longer sequences of tight passages, where side

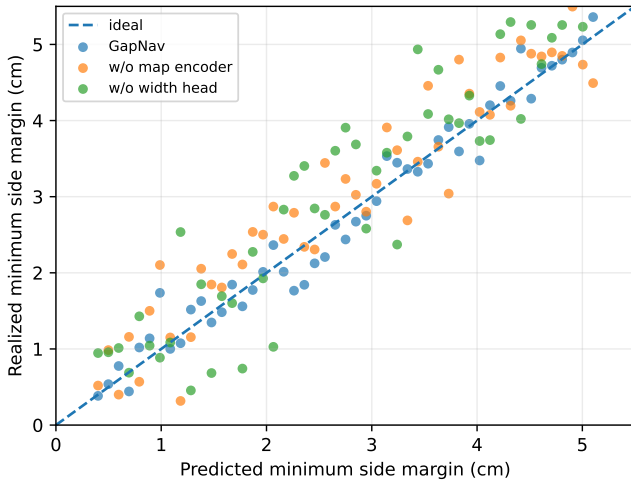


Fig. 6: Predicted versus realized minimum side margin. GapNav remains closest to the ideal diagonal and shows lower bias and spread than the baseline variants.

TABLE III: Hardware evaluation on a physical quadruped using the same high-level passage-mode policy. Each row aggregates 45 trials.

Side margin	Success (%)	Entry precision (%)	Mean time (s)
2 cm / side	46.7	71.1	5.34
4 cm / side	73.3	86.7	4.92
6 cm / side	88.9	93.3	4.58
8 cm / side	93.3	95.6	4.36

margin must be managed repeatedly rather than only once.

ACKNOWLEDGMENTS

Fig. 1 is annotated with an AI editor.

REFERENCES

- [1] Jemin Hwangbo, Joonho Lee, Alexey Dosovitskiy, C. Dario Bellicoso, Vassilios Tsounis, Vladlen Koltun, and Marco Hutter. Learning agile and dynamic motor skills for legged robots. *Science Robotics*, 4(26), 2019.
- [2] Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning quadrupedal locomotion over challenging terrain. *Science Robotics*, 5(47), 2020.
- [3] Ashish Kumar, Zipeng Fu, Deepak Pathak, and Jitendra Malik. RMA: Rapid motor adaptation for legged robots. In *Proceedings of Robotics: Science and Systems*, 2021.
- [4] Nikita Rudin, David Hoeller, Philipp Reist, and Marco Hutter. Learning to walk in minutes using massively parallel deep reinforcement learning. In *Proceedings of the 5th Conference on Robot Learning*, volume 164 of *Proceedings of Machine Learning Research*, pages 91–100. PMLR, 2022.
- [5] Russell Buchanan, Tirthankar Bandyopadhyay, Marko Bjelonic, Lorenz Wellhausen, Marco Hutter, and Navinda Kottege. Walking posture adaptation for legged robot navigation in confined spaces. *IEEE Robotics and Automation Letters*, 4(2):2148–2155, 2019.
- [6] Russell Buchanan, Lorenz Wellhausen, Marko Bjelonic, Tirthankar Bandyopadhyay, Navinda Kottege, and Marco Hutter. Perceptive whole-body planning for multilegged robots in confined spaces. *Journal of Field Robotics*, 38(1):68–84, 2021.
- [7] Brendan Tidd, Akansel Cosgun, Jürgen Leitner, and Nicolas Hudson. Passing through narrow gaps with deep reinforcement learning. In *2021 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 3492–3498. IEEE, 2021.

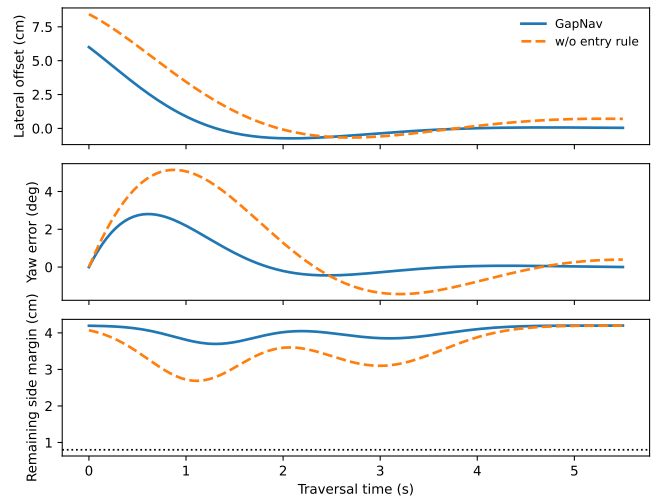


Fig. 7: Hardware traversal traces for a representative trial. GapNav corrects lateral offset and yaw more quickly and keeps the remaining side margin above the safety threshold during the crossing.

- [8] Takahiro Miki, Joonho Lee, Lorenz Wellhausen, and Marco Hutter. Learning to walk in confined spaces using 3d representation. arXiv:2403.00187, 2024.
- [9] Qiayuan Liao, Zhongyu Li, Akshay Thirugnanam, Jun Zeng, and Koushil Sreenath. Walking in narrow spaces: Safety-critical locomotion control for quadrupedal robots with duality-based optimization. In *2023 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2723–2730, 2023.
- [10] Ruben Grandia, Andrew J. Taylor, Aaron D. Ames, and Marco Hutter. Multi-layered safety for legged robots via control barrier functions and model predictive control. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, 2021.
- [11] Abdelrahman Abdalla, Michele Focchi, Romeo Orsolino, and Claudio Semini. An efficient paradigm for feasibility guarantees in legged locomotion. *IEEE Transactions on Robotics*, 39(5):3499–3515, 2023.
- [12] Jie Tan, Tingnan Zhang, Erwin Coumans, Atil Iscen, Yunfei Bai, Danijar Hafner, Steven Bohez, and Vincent Vanhoucke. Sim-to-real: Learning agile locomotion for quadrupedal robots. In *Proceedings of Robotics: Science and Systems*, Pittsburgh, Pennsylvania, June 2018.
- [13] Zifan Xu, Amir Hossain Raj, Xuesu Xiao, and Peter Stone. Dexterous legged locomotion in confined 3d spaces with reinforcement learning. In *2024 IEEE International Conference on Robotics and Automation (ICRA)*, pages 11474–11480. IEEE, 2024.
- [14] Jing Guo, Ziwei Wang, and Weibang Bai. Learning quadrupedal robot locomotion for narrow pipe inspection. In *2025 IEEE International Conference on Robotics and Biomimetics (ROBIO)*, pages 555–561. IEEE, 2025.
- [15] Ananye Agarwal, Ashish Kumar, Jitendra Malik, and Deepak Pathak. Legged locomotion in challenging terrains using egocentric vision. In *Conference on robot learning*, pages 403–415. PMLR, 2023.
- [16] David Hoeller, Nikita Rudin, Dhionis Sako, and Marco Hutter. ANYmal Parkour: Learning agile navigation for quadrupedal robots. *Science Robotics*, 9(88):eadi7566, 2024.
- [17] Takahiro Miki, Joonho Lee, Jemin Hwangbo, Lorenz Wellhausen, Vladlen Koltun, and Marco Hutter. Learning robust perceptive locomotion for quadrupedal robots in the wild. *Science Robotics*, 7(62), 2022.
- [18] Shamel Fahmi, Victor Barasuol, Domingo Esteban, Octavio Villarreal, and Claudio Semini. VITAL: Vision-based terrain-aware locomotion for legged robots. *IEEE Transactions on Robotics*, 39(2), 2023.
- [19] Yinzhaodong, Ji Ma, Liu Zhao, Wanyue Li, and Peng Lu. MARG: Mastering risky gap terrains for legged robots with elevation mapping. *IEEE Transactions on Robotics*, 41:6123–6139, 2025.
- [20] Xuxin Cheng, Kexin Shi, Ananye Agarwal, and Deepak Pathak. Extreme parkour with legged robots. In *2024 IEEE International*

- Conference on Robotics and Automation (ICRA)*, pages 11443–11450. IEEE, 2024.
- [21] Takahiro Miki, Lorenz Wellhausen, Ruben Grandia, Fabian Jenelten, Timon Homberger, and Marco Hutter. Elevation mapping for locomotion and navigation using gpu. In *2022 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pages 2273–2280. IEEE, 2022.
 - [22] Magnus Gaertner, Marko Bjelonic, Farbod Farshidian, and Marco Hutter. Collision-free mpc for legged robots in static and dynamic scenes. In *2021 IEEE International Conference on Robotics and Automation (ICRA)*, pages 8266–8272. IEEE, 2021.
 - [23] Fabian Jenelten, Junzhe He, Farbod Farshidian, and Marco Hutter. DTC: Deep tracking control. *Science Robotics*, 2024.
 - [24] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms. *arXiv preprint arXiv:1707.06347*, 2017.